

## **14.3 Creating and Working with Generic Classes**

### **👉 Using Generics in Custom Classes**

We have previously seen how generics are used in collections to ensure type safety. Now, we will explore how to apply generics to custom classes, allowing us to take advantage of type safety in our own implementations. This approach will enable better code management and reduce errors as we proceed.

---

### **👉 What is a Generic Class?**

A generic class is a class that can refer to any data type. In generic classes, we use a type parameter (commonly denoted as T) to specify the type at the time of object creation. This helps create flexible and type-safe classes.

---

### **👉 Example 1: Creating a Generic Class**

Let's look at an example where we create a generic class named Student. We will define it using a type parameter T to ensure type safety:

```
// Generic Student class with a type parameter T
class Student<T> {
    T obj; // Reference type for classes like Integer, Float,
String, etc.

    // Constructor to initialize the object
    public Student(T obj) {
        this.obj = obj;
    }

    // Method to display the type of the data used
    public void display() {
        System.out.println("The type of data used here is: " +
obj.getClass().getName());
    }

    // Method to get the value of the object
    public T getObj() {
        return obj;
    }
}
```

```

}

// Demonstration of the Student class
public class Demo {
    public static void main(String[] args) {
        // Using generics to create a Student object of type
Integer
        Student<Integer> st1 = new Student<>(10);
        st1.display();
        System.out.println(st1.getObj());

        // Using generics to create a Student object of type
String
        Student<String> st2 = new Student<>("Rohan");
        st2.display();
        System.out.println(st2.getObj());
    }
}

```

## Output

```

The type of data used here is: java.lang.Integer
10
The type of data used here is: java.lang.String
Rohan
|

```

## Explanation

In the example above:

1. **Generic Class Definition:** We define the Student class using a generic type parameter T. This allows us to specify the data type when creating an object, ensuring type safety. For example, Student<Integer> uses an Integer type, and Student<String> uses a String type.
2. **Type Safety:** Using generics ensures that we work with the specified data type only, reducing the chance of runtime errors. If we had not used generics, the IDE would have shown warnings about potential type safety issues.

3. **Display Method:** The `display()` method shows the name of the class type used for the generic parameter (T). This provides clarity on the data type being utilized.
4. **Getter Method (`getObj()`):** The method returns the object of type T. The value assigned during object creation is returned in its original type, thanks to the use of generics.

This example demonstrates how generics can be applied to custom classes to achieve the same type safety that collections provide.

---

### Using a Generic Method

Just like a generic class, we can create generic methods that accept any type of argument. The scope of the type parameter is limited to the method where it is declared, allowing for both static and non-static generic methods.

---

### Common Type Parameters in Generics

When working with generics, it's essential to follow standard naming conventions for type parameters:

- **T** - Type
- **E** - Element (used commonly in collections)
- **K** - Key (used in maps)
- **N** - Number
- **V** - Value (used in maps)

Following these conventions helps make the code more readable and consistent.

---

### Conclusion

- Generics allow us to create classes and methods that are flexible and type-safe.
- By specifying a type parameter, we can avoid type-casting and related errors, making our code more robust.

- Generic classes and methods offer the same advantages in custom implementations as they do in collections.